

SkillSphere

Project Documentation — Onboarding Guide for Team Members

Last updated: July 9, 2026

This document is meant to be self-contained: a teammate who has never seen the SkillSphere project before should be able to read this file, top to bottom, and understand what the project is, how it is built, what already works, and exactly what to do next — without needing to ask anyone else first.

1. What is SkillSphere?

SkillSphere is a learning platform website, similar in spirit to Coursera or Udemy. Users sign in with their Google account, browse courses, and (eventually) track their learning progress. The current landing page uses a custom “Skill Trail” expedition theme, where courses are presented as hiking “routes”, features are “basecamp instruments”, and progress/analytics is a “trail log”.

The original requirement was: a landing page, a login page that uses Google Sign-In, and a backend with JWT authentication and a database containing a users table. That original scope is fully built and verified working (see Section 6).

2. Tech Stack

Layer	Technology	Notes
Frontend	React (Vite)	JavaScript, not TypeScript
Backend	Java 17 + Spring Boot 3.5.16	Maven build (mvnw)
Database	MySQL	Local MySQL during development; Aiven cloud MySQL migration planned
Authentication	Google OAuth2 (OIDC) + JWT	Login via Google; JWT used for API session/auth
Frontend routing	react-router-dom	Added to handle the /oauth-success callback page

Why this stack: backend language was locked to Java early on, so Spring Boot was chosen because it has first-class, well-documented support for exactly this combination — Google OAuth2 login, JWT authentication, and MySQL via Spring Data JPA/Hibernate. React (Vite) was chosen for the frontend because it pairs cleanly with a Spring Boot REST API and has abundant tutorials for “Google login + JWT” flows.

3. High-Level Architecture

There are three moving parts:

- Frontend (client/) — the React app the user sees: landing page, “Continue with Google” button, and a small page that catches the login result.
- Backend (server/) — a Spring Boot app that talks to Google, issues JWT tokens, and exposes protected API endpoints.
- Database (MySQL) — stores a single users table right now (id, email, google_id, name, profile_picture), auto-created by Hibernate from the User entity.

3.1 How login works, step by step

1. User clicks “Continue with Google” on the React landing page.
2. Browser is redirected to Google’s real sign-in page (this is standard OAuth2/OIDC, not a custom login form).
3. User logs in and consents; Google redirects back to the backend at `/login/oauth2/code/google` with an authorization code.
4. Spring Security exchanges that code with Google for the user’s profile (email, name, Google ID, picture).
5. CustomOAuth2UserService checks the users table: if the email already exists, it reuses that row; otherwise it creates a new user row.
6. OAuth2LoginSuccessHandler generates a JWT for that user and redirects the browser to the frontend at `http://localhost:5173/oauth-success?token=<jwt>`.
7. The React OAuthSuccess.jsx page reads the token from the URL, stores it (via AuthContext, in sessionStorage), and redirects the user back to the homepage as “logged in”.
8. From then on, the frontend sends that JWT in the `Authorization: Bearer <token>` header when calling protected backend endpoints. JwtAuthenticationFilter validates it on every request.

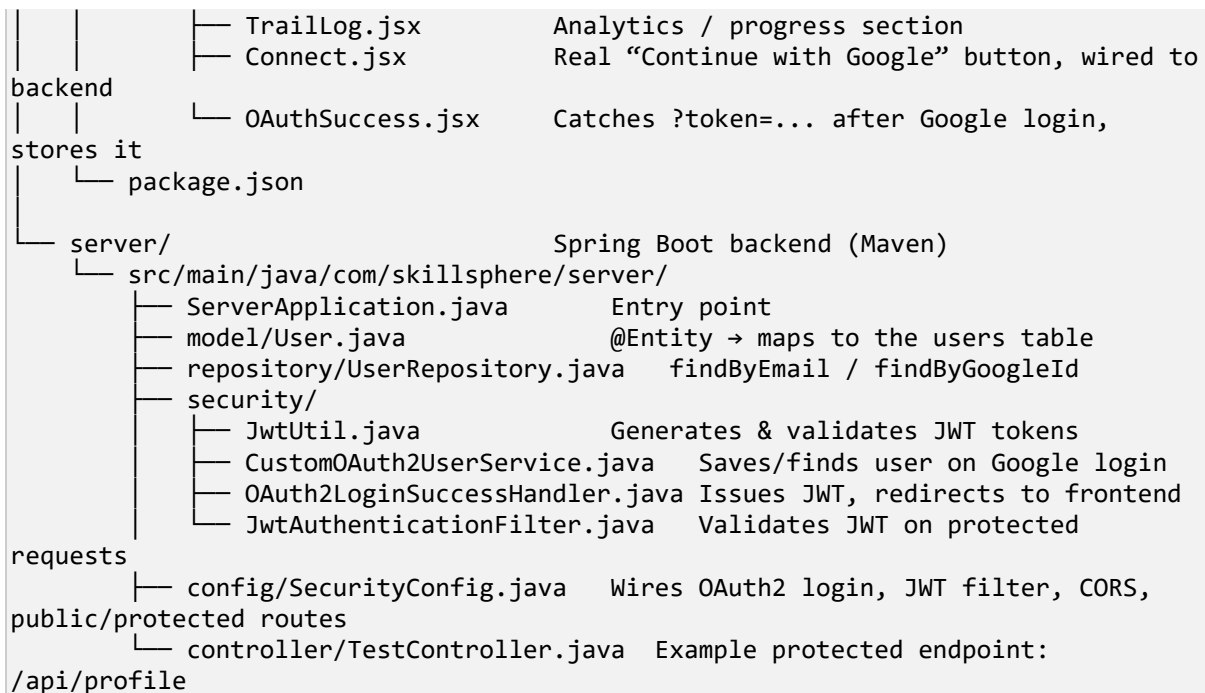
This entire flow has been tested end-to-end (Google login → user saved in MySQL → JWT issued → protected endpoint accepts the token and rejects requests without one). See Section 6 for the exact verification results.

4. Project Folder Structure

```

skillsphere/ (repo root, e.g. TeamC_SkillSphere-learning-nexus-fsd/)
├── client/ React (Vite) frontend
│   ├── src/
│   │   ├── App.jsx Routes + wraps app in AuthProvider
│   │   ├── index.css Global “Skill Trail” theme styles
│   │   ├── main.jsx Vite/React entry point (untouched default)
│   │   └── context/
│   │       └── AuthContext.jsx Global login-state (token, login(),
│   │           logout())
│   └── components/
│       ├── Navbar.jsx Top nav, shows Sign in / Sign out
│       ├── Hero.jsx Landing hero section (“Explore”)
│       ├── Routes.jsx Courses shown as trail “routes” (“Learn”)
│       └── Instruments.jsx Feature list, “basecamp instruments”
└── (“Track”)

```



5. Backend Details (server/)

5.1 Files and what each one does

File	Purpose
model/User.java	JPA entity. Defines the users table columns: id, email, google_id, name, profile_picture.
repository/UserRepository.java	Spring Data JPA interface. Gives free save()/findAll()/etc., plus custom finders findByEmail and findById (Spring auto-generates the SQL from the method name).
security/JwtUtil.java	Creates a signed JWT for a given email and can validate/parse a token later.
security/CustomOAuth2UserService.java	Runs right after Google confirms the user’s identity. Reads email/name/googleId/picture from Google’s response, and either finds the existing user row or creates a new one.
security/OAuth2LoginSuccessHandler.java	Runs immediately after a successful Google login. Generates a JWT via JwtUtil and redirects the browser to the frontend’s /oauth-success page with the token in the URL.
security/JwtAuthenticationFilter.java	Checks the Authorization: Bearer <token> header on incoming API requests and validates the JWT before letting the request through.
config/SecurityConfig.java	Central security wiring: which routes are public vs. need a valid JWT, CORS settings (allows the frontend on localhost:5173 to call the backend on localhost:8080), and plugs in the OAuth2 login flow with the custom classes above.
controller/TestController.java	A simple protected GET /api/profile endpoint used to prove the whole chain works.

5.2 application.properties (config)

This file lives at `server/src/main/resources/application.properties` and is intentionally excluded from Git (see Section 9) because it holds real secrets. It is not present when you clone the repository — every teammate must create it locally. Use this template and fill in the real values (get them from whoever set up the Google Cloud project / database, do not reuse the ones that may have been pasted in old chat history, since those should be rotated):

```
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/skillsphere_db
spring.datasource.username=root
spring.datasource.password=<your_local_mysql_password>
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

# Google OAuth2 Configuration
spring.security.oauth2.client.registration.google.client-id=<from Google Cloud Console>
spring.security.oauth2.client.registration.google.client-secret=<from Google Cloud Console>
spring.security.oauth2.client.registration.google.scope=openid,profile,email

# JWT Configuration
jwt.secret=<any long random string>
jwt.expiration=86400000
```

The Google Client ID / Secret come from Google Cloud Console → “Google Auth Platform” → Clients → the Skillsphere Web Client. The configured redirect URI there is `http://localhost:8080/login/oauth2/code/google` — this must match exactly or login will fail with a 401 during the token exchange step.

5.3 Database

Database name: `skillsphere_db` (MySQL). You need MySQL installed locally and a database with this name created before the backend will start successfully. Hibernate (`spring.jpa.hibernate.ddl-auto=update`) automatically creates/updates the users table from `User.java` — no manual SQL is required.

6. Frontend Details (client/)

6.1 Design

Theme: “Skill Trail” expedition concept — deep forest-ink background, trail-orange accent, sage highlights. Courses are shown as “routes” with elevation/distance-style stats, features are shown as “basecamp instruments”, and progress/analytics is a “trail log” with an elevation-profile-style chart. This was a deliberate choice to avoid generic dark sci-fi or cream/serif/terracotta looks.

6.2 Auth state on the frontend

context/AuthContext.jsx holds the JWT in React state and mirrors it into sessionStorage (chosen over localStorage so the login doesn't persist forever — it clears when the browser tab closes, which is a reasonable middle ground for a student project). It exposes login(token), logout(), token, and isAuthenticated. Navbar.jsx and Connect.jsx both read this context to show “Sign in” vs. “Sign out”.

components/OAuthSuccess.jsx is the page Google/the backend redirects to after a successful login (/oauth-success?token=...). It reads the token out of the URL query string, calls login(token) from AuthContext, and redirects the user back to the homepage.

6.3 Running the frontend

```
cd client
npm install
npm run dev
```

Then open <http://localhost:5173> in the browser. The backend (Section 5) must also be running for the “Continue with Google” button to work, since it calls the real backend OAuth endpoint.

7. Current Status

7.1 Fully working and verified

- Google OAuth2 login end-to-end (redirect to Google → consent → redirect back).
- New users are correctly created in MySQL; returning users are correctly recognized by email.
- JWT is generated on successful login and passed to the frontend via the redirect URL.
- Protected backend endpoint (/api/profile) returns 200 OK with a valid token and blocks requests without one.
- Frontend catches the JWT after login, stores it, and reflects logged-in/out state in the UI (Navbar and Connect sections).
- “Skill Trail” themed landing page is built and rendering correctly in the browser.

7.2 Not done yet / next steps

- Migrate the database from local MySQL to a cloud MySQL instance (Aiven was the plan) — includes handling the SSL connection that Aiven requires, unlike local MySQL.
- Build out real course content/pages beyond the landing page (the “Routes” section currently shows placeholder/sample courses).
- Consider returning a clean 401 JSON response for unauthenticated API calls instead of the current default behavior (redirecting to the Google login page) — this matters more once there's a real API-driven frontend beyond the login flow.
- Rotate the Google OAuth client secret, JWT secret, and MySQL password if any of the values that were shared in chat/screenshots during development are still in use — treat anything that appeared in a chat or screenshot as compromised.

8. Setup Guide — Running the Project from Scratch

For a teammate setting this up for the first time:

Step 1 — Clone the repo and install prerequisites

- Install: Java 17 (or newer), Node.js, MySQL, Maven is not required separately — the project includes the Maven wrapper (mvnw).
- Clone the repository and open it in your editor of choice (the project was built using IntelliJ IDEA for the backend).

Step 2 — Database

- Start MySQL locally and create a database named `skillsphere_db` (e.g. via MySQL Workbench, or `CREATE DATABASE skillsphere_db;`).

Step 3 — Backend config

- Create `server/src/main/resources/application.properties` using the template in Section 5.2, filled in with real values (ask a teammate for the Google OAuth credentials and DB password; do not commit this file).
- Run the backend: open `ServerApplication.java` and run it (or `./mvnw spring-boot:run` from `server/`). It should log “Tomcat started on port(s): 8080” and “Started ServerApplication” with no errors.

Step 4 — Frontend

- From `client/`, run `npm install` then `npm run dev`, and open `http://localhost:5173`.

Step 5 — Verify login works

- Click “Continue with Google” on the homepage, sign in with a Google account that has been added as a test user in the Google Cloud OAuth consent screen (the app is currently in “testing” mode, so only approved test users can log in), and confirm you land back on the homepage in a “signed in” state.

9. Git Notes

A `.gitignore` is set up at the repo root to exclude build artifacts and secrets:

```
# Node
node_modules/
dist/
.vite/

# Java / Maven
target/
*.class

# IDE
.idea/
*.iml

# Secrets
.env
application.properties
```

Because `application.properties` is git-ignored, it will never appear when you clone the repo — you must recreate it locally every time using Section 5.2’s template. This is intentional: it keeps the Google client secret, JWT secret, and DB password out of the shared repository.

10. Troubleshooting Notes from Development

These are real issues hit while building this project — useful if a teammate runs into the same thing:

Symptom	Cause / Fix
401 Unauthorized during Google token exchange	Usually a mismatched/incorrect Google Client Secret. Fix: regenerate the secret in Google Cloud Console and paste it directly (no retyping) into <code>application.properties</code> .
“Cannot resolve symbol JpaRepository” in IntelliJ despite Maven having the dependency	Stale IDE index. Fix: File → Invalidate Caches → check “Clear file system cache and Local History” → Invalidate and Restart.
Hitting / or /api/profile in Postman redirects to Google’s sign-in HTML instead of returning JSON	Expected default behavior: Spring Security’s OAuth2 login redirects unauthenticated requests to Google rather than returning a plain 401. Not a bug — confirms the route is protected.
Extra nested folder after generating the Spring Boot project from start.spring.io (server/server/...)	The zip contents were extracted into an already-existing server folder. Fix: move <code>pom.xml</code> , <code>src/</code> , <code>.mvn/</code> , etc. up so they sit directly inside <code>server/</code> , then delete the empty nested folder.
<code>.gitignore</code> file was silently saved as <code>.gitignore.txt</code>	Windows can hide the <code>.txt</code> extension. Fix: enable “File name extensions” in File Explorer’s View menu, then rename to remove <code>.txt</code> .
Cloned the repo fresh and the backend won’t start (“Failed to determine a suitable driver class”)	<code>application.properties</code> is git-ignored, so it doesn’t exist in a fresh clone. Recreate it locally (Section 5.2).

11. Glossary (plain-English)

Term	In plain words
JWT (JSON Web Token)	A digital “wristband” the backend hands the frontend after a successful login. It proves “this person is verified” without needing to log in again on every request.
OAuth2 / OIDC	The standard way to let users “Sign in with Google” without your app ever seeing their Google password.
Spring Security	The part of the backend that decides which pages/API calls are public vs. require a valid login.
Hibernate / JPA	The tool that turns a Java class (like <code>User.java</code>) into a real MySQL table automatically, so you don’t write raw SQL for basic operations.
CORS	A browser security rule that would normally block the frontend (port 5173) from talking to the backend (port 8080) unless the backend explicitly allows it — configured in <code>SecurityConfig.java</code> .

If anything in this document is unclear or out of date, update it directly rather than re-explaining verbally — that keeps this file trustworthy as the single source of truth for the project.